

Experiment 9: A Search Algorithm*

Aim

To implement the A* Search algorithm for finding the shortest path between two nodes.

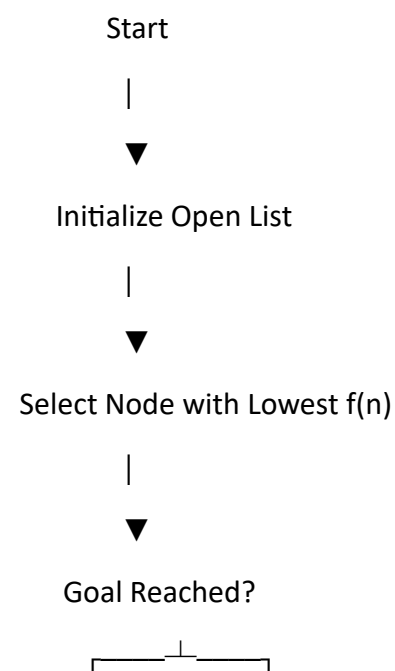
Objective

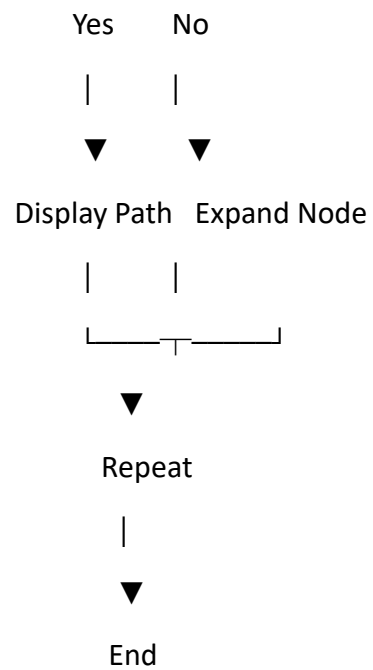
To find the optimal path using both the actual path cost and heuristic value.

Algorithm

1. Initialize the open and closed lists.
2. Insert the start node into the open list.
3. Select the node with the lowest $f(n) = g(n) + h(n)$.
4. Expand the selected node.
5. Update the cost of neighbouring nodes.
6. Repeat until the goal node is reached.
7. Display the shortest path.

Flowchart





Python Code

```
from queue import PriorityQueue
```

```
graph = {  
    'A': {'B': 1, 'C': 4},  
    'B': {'D': 2, 'E': 5},  
    'C': {'F': 3},  
    'D': {},  
    'E': {'F': 1},  
    'F': {}  
}
```

```
heuristic = {  
    'A': 7,  
    'B': 6,  
    'C': 2,  
    'D': 1,
```

```
'E': 1,  
'F': 0  
}
```

```
def astar(start, goal):  
    pq = PriorityQueue()  
    pq.put((0, start))  
    cost = {start: 0}  
  
    while not pq.empty():  
        _, node = pq.get()  
  
        if node == goal:  
            print("Goal Reached")  
            return  
  
        for neighbour in graph[node]:  
            new_cost = cost[node] + graph[node][neighbour]  
  
            if neighbour not in cost or new_cost < cost[neighbour]:  
                cost[neighbour] = new_cost  
                priority = new_cost + heuristic[neighbour]  
                pq.put((priority, neighbour))  
  
    astar('A', 'F')
```

Sample Output

Goal Reached

Result

Thus, the A* Search algorithm was successfully implemented to find the optimal path between the source and destination nodes.